UNIWERSYTET WSB MERITO WROCŁAW

Bezpieczeństwo Aplikacji

Wykład 1: Krajobraz zagrożeń, anatomia podatności i automatyzacja testów bezpieczeństwa

Skrypt wykładowy dla prowadzącego i studentów

Prowadzący: mgr Przemysław Żarnecki
Uczelnia: Uniwersytet WSB Merito we Wrocławiu
Kierunek: Informatyka, Studia II stopnia (Semestr zimowy 2026/2027)

Repozytorium Git:
`https://dev.zarnecki.org/Merito/bezpieczenstwo_aplikacji.git`
Materiały dydaktyczne (Nextcloud):
`https://chmura.zarnecki.org/s/iLdT3J5YGTTYCPr`

Wrocław, Październik 2026

Spis treści

1 Wprowadzenie: iluzja bezpieczeństwa i przypadek ekspresu do kawy 2
1.1 Współczesne wpadki bezpieczeństwa aplikacji (2025/2026) 2

2 Dlaczego aplikacje są głównym celem ataków? 4
2.1 Słowniczek kluczowych pojęć i skrótów 4

3 OWASP Top 10 (Edycja 2025): Ewolucja zagrożeń 4

4 Modelowanie zagrożeń (Threat Modeling) 6

5 Automatyzacja testów bezpieczeństwa: SAST, DAST, SCA 7
5.1 1. SAST (Static Application Security Testing) 7
5.2 2. DAST (Dynamic Application Security Testing) 7
5.3 3. SCA (Software Composition Analysis) 7

6 Fundamenty architektoniczne: Zasady Secure by Design 8

7 Materiały dydaktyczne i repozytorium 8

1. Wprowadzenie: iluzja bezpieczeństwa i przypadek ekspresu do kawy

Tradycyjne postrzeganie bezpieczeństwa IT przez dekady opierało się na koncepcji *zamku i fosy* (ang. *perimeter security*). Zakładano, że wystarczy odgrodzić sieć firmową zaporą ogniową (firewall), a wszystkie aplikacje i urządzenia znajdujące się wewnątrz mogą sobie bezgranicznie ufać.

Współczesne podejście do architektury i wytwarzania oprogramowania całkowicie zdezaktualizowało ten model. Aplikacje nie są już zamkniętymi monolitami działającymi na odizolowanych serwerach. Są rozproszonymi ekosystemami komunikującymi się przez REST API, WebSocket, kolejki zdarzeń oraz usługi chmurowe, integrując dziesiątki bibliotek open-source i komunikując się z urządzeniami brzegowymi.

Studium przypadku: Inteligentny ekspres, który przesłał 1 TB danych

W październiku 2026 roku serwisy technologiczne (m.in. ITHardware.pl) opisały incydent z inteligentnym ekspresem do kawy marki Keurig podłączonym do domowej sieci Wi-Fi. Właściciel urządzenia, przeglądając statystyki przepływu danych na domowym routerze brzegowym, zauważył anomalną aktywność: w ciągu zaledwie 10 dni ekspres wysłał do Internetu **około 1 TB danych** oraz pobrał niemal 10 GB.

Przyczyna techniczna i wnioski architektoniczne:

Ekspres do kawy nie przetwarzał terabajtów danych o parzeniu kawy. Urządzenie posiadało wbudowany miniaturowy system operacyjny z pełnym stosem sieciowym Linux, podatny firmware, otwarte porty serwisowe oraz domyślne dane logowania. Wystawione na świat API pozwoliło napastnikom przejąć kontrolę nad procesorem urządzenia i włączyć je w skład botnetu (wykorzystywanego do ataków DDoS, pośrednictwa w nielegalnym ruchu proxy lub dystrybucji złośliwego oprogramowania).

Wniosek techniczny: Dla atakującego nie ma znaczenia, czy celem jest serwer bazodanowy banku, panel e-commerce, czy ekspres do kawy w kuchni. Każdy komponent oprogramowania posiadający interfejs sieciowy, który został napisany bez kontroli wejścia, z twardo zakodowanymi poświadczeniami i niezałatany, staje się natychmiastowym punktem zaczepienia (*Initial Foothold*).

1.1. Współczesne wpadki bezpieczeństwa aplikacji (2025/2026)

Incydent z ekspresem to nie odosobniony kuriozalny przypadek, lecz symptom szerszego zjawiska. Błędy bezpieczeństwa dotykają wszystkich kategorii współczesnego

oprogramowania:

- **Aplikacje mobilne (Android/iOS) – wycieki baz chmurowych i Deep Links:** W latach 2025–2026 audyty ujawniły ponad 150 popularnych aplikacji mobilnych pozostawiających otwarte instancje Firebase oraz Supabase bez wymaganych reguł autoryzacji (*Security Rules*). Twardo zakodowane w plikach APK/IPA klucze konfiguracyjne pozwalały na bezpośredni odczyt i modyfikację baz danych klientów. Równolegle poważnym wektorem pozostają błędy obsługi Deep Links oraz Android Intent Spoofing, umożliwiające przechwytywanie sesji i tokenów OAuth.
- **Aplikacje desktopowe (Electron / WebView / Tauri) – RCE przez schematy URL:** Powszechność technologii webowych na desktopie przynosi unikalne ryzyka. Błędy konfiguracji (np. flagi `nodeIntegration: true` czy wyłączona `contextIsolation`) w połączeniu z rejestracją niestandardowych schematów URL (np. `app://` lub `slack://`) pozwalały napastnikom na wykonanie dowolnego kodu w systemie operacyjnym (RCE) po kliknięciu przez ofiarę w spreparowany link na stronie internetowej.
- **Aplikacje chmurowe i łańcuch dostaw (Supply Chain):** Incydenty takie jak przejęcie domeny Polyfill.io (wstrzyknięcie złośliwego skryptu do setek tysięcy serwisów na całym świecie) czy podatności w potokach CI/CD dobitnie pokazały, że współczesne aplikacje dziedziczą podatności każdego importowanego skryptu i biblioteki trzeciej.

Lista Grozy: 50 spektakularnych incydentów (2020–2026)

W interaktywnej prezentacji wykładowej przygotowano szczegółową bazę 50 rzeczywistych kompromitacji oprogramowania z ostatnich lat, podzielonych na 5 kluczowych filarów architektury:

- **Aplikacje mobilne (10):** m.in. TikTok (Deep Links JS Bridge), WhatsApp (Zero-Click RCE), Telegram (EvilVideo), 23andMe, Duolingo, Strava.
- **Aplikacje desktopowe (10):** m.in. Discord i Slack (Electron RCE / Token Theft), Zoom (NTLM UNC Injection), AnyDesk (kompromitacja certyfikatów), KeePass (zrzut RAM), WinRAR (CVE-2023-38831).
- **Web & SaaS (10):** m.in. LastPass (atak boczny przez podatność w Plex), Okta (wyciek plików HAR), CircleCI, Snowflake / Ticketmaster (brak wymuszenia MFA), Atlassian Confluence.
- **Infrastruktura & Chmura (10):** m.in. Microsoft Exchange (ProxyLogon), Outlook (CVE-2023-23397), Citrix Bleed, Ivanti Connect Secure, MOVEit Transfer (SQLi).
- **Łańcuch dostaw (10):** m.in. XZ Utils (backdoor w liblzma), Log4Shell, Polyfill.io, CrowdStrike Falcon (Channel File 291), PyTorch (Dependency Confusion),

Ledger Connect Kit.

2. Dlaczego aplikacje są głównym celem ataków?

Współczesne ataki komputerowe w przeważającej większości nie polegają na przełamywaniu zaawansowanej kryptografii AES-256 czy szukaniu podatności w protokole TCP/IP. Celem numer jeden stała się **warstwa aplikacji** (warstwa 7 modelu OSI).

Przyczyny tego stanu rzeczy wynikają ze specyfiki nowoczesnych architektur systemowych:

1. **Eksplozywny wzrost złożoności kodu:** Typowa aplikacja webowa składa się dziś w 10–20% z kodu autorskiego oraz w 80–90% z zewnętrznych bibliotek, modułów i kontenerów (ekosystemy npm, PyPI, Maven, Docker Hub).
2. **Dostępność z dowolnego miejsca:** Porty 80 i 443 muszą być z definicji otwarte na cały świat, co omija tradycyjne zapory sieciowe.
3. **Błędy w logice biznesowej:** Zapory WAF (Web Application Firewall) potrafią zablokować proste sygnatury SQLi, ale nie są w stanie zrozumieć, że użytkownik o ID 42 nie powinien wyświetlać faktury użytkownika o ID 43 (błędy typu IDOR/BOLA).
4. **Dług technologiczny i presja czasu:** Funkcjonalności biznesowe (*Time to Market*) są niemal zawsze priorytetyzowane ponad bezpieczeństwo, a audyty kodu przeprowadza się zbyt późno.
5. **Generowanie kodu przez sztuczną inteligencję (AI) i halucynacje pakietów:** Powszechne korzystanie z asystentów AI (GitHub Copilot, ChatGPT, Claude) wprowadza nowe zagrożenia: bezkrytyczne wklejanie niesprawdzonego kodu zawierającego luki oraz tzw. *AI Package Hallucination* (sugerowanie importów nieistniejących pakietów, które są następnie rejestrowane przez przestępców w ekosystemach npm czy PyPI ze złośliwą zawartością).

2.1. Słowniczek kluczowych pojęć i skrótów

3. OWASP Top 10 (Edycja 2025): Ewolucja zagrożeń

Projekt Open Web Application Security Project (OWASP) od ponad dwóch dekad publikuje zestawienie najbardziej krytycznych zagrożeń bezpieczeństwa aplikacji webowych. Zestawienie OWASP Top 10:2025 odzwierciedla głęboką zmianę we współczesnych architekturach:

Skrót	Pełna nazwa	Znaczenie w kontekście bezpieczeństwa
L7	Layer 7 (Application Layer)	Najwyższa warstwa modelu OSI (HTTP/HTTPS, WebSocket, gRPC). Cel większości nowoczesnych ataków.
WAF	Web Application Firewall	Zapora aplikacyjna analizująca ruch HTTP. Blokuje znane sygnatury ataków, lecz nie rozumie logiki biznesowej.
IDOR	Insecure Direct Object Reference	Brak autoryzacji przy bezpośrednim odwołaniu do identyfikatora obiektu (np. `/user/105`).
BOLA	Broken Object Level Auth.	Nowoczesna nazwa podatności IDOR w specyfikacji OWASP API Security Top 10.
ORM	Object-Relational Mapping	Warstwa mapowania obiektowo-relacyjnego (np. SQLAlchemy, Hibernate). Chroni przed SQLi przez parametryzację.
CVE	Common Vulnerabilities & Exposures	Międzynarodowy rejestr powszechnie znanych i skatalogowanych podatności w oprogramowaniu.
SBOM	Software Bill of Materials	Formalna lista wszystkich komponentów, bibliotek i modułów wchodzących w skład aplikacji.

Tabela 1: Słowniczek podstawowych pojęć bezpieczeństwa warstwy aplikacji.

Pozycja	Kategoria OWASP 2025	Esencja problemu technicznego
A01	Broken Access Control	Błędy autoryzacji horyzontalnej i wertykalnej (IDOR, BOLA, SSRF włączone do A01).
A02	Security Misconfiguration	Domyślne hasła, publiczne zasobniki chmurowe (S3), zbędne otwarte porty, brak nagłówków.
A03	Software Supply Chain Failures	Podatne, przestarzałe lub zainfekowane pakiety i zależności w cyklu wytwórczym.
A04	Cryptographic Failures	Brak szyfrowania danych wrażliwych w spoczynku i w tranzycie, szybkie skróty haseł (MD5).
A05	Injection	Wstrzyknięcia SQL, NoSQL, OS Command, XSS wynikające z braku separacji kodu od danych.
A06	Insecure Design	Błędy na etapie planowania architektury, brak modelowania zagrożeń STRIDE.
A07	Authentication Failures	Słabe hasła, brak ochrony przed brute-force, podatność 2FA na phishing AiTM.
A08	Software or Data Integrity	Niezabezpieczone aktualizacje, deserializacja danych z niezaufanych źródeł, manipulacja kodem.
A09	Logging & Alerting Failures	Brak logowania zdarzeń krytycznych, brak powiadomień o trwających anomaliach.
A10	Mishandling of Exceptional Conditions	Nieprawidłowa obsługa błędów ujawniająca ścieżki i zrzuty pamięci (StackTrace).

Tabela 2: Kategorie zagrożeń wg standardu OWASP Top 10:2025.

Ważna uwaga analityczna

Zwróć uwagę na spadek pozycji klasycznych wstrzyknięć (**A05 Injection**) na rzecz błędów kontroli dostępu (**A01**) oraz problemów łańcucha dostaw (**A03**). Współczesne frameworki programistyczne (ORM-y, silniki szablonów z automatycznym escapowaniem) w znacznym stopniu rozwiązują problem prostych wstrzyknięć. Jednak żaden framework nie wie sam z siebie, czy dany użytkownik ma prawo edytować dany rekord w bazie – to zależy wyłącznie od poprawnej logiki autoryzacji zaimplementowanej przez programistę.

4. Modelowanie zagrożeń (Threat Modeling)

Bezpieczeństwo aplikacji zaczyna się przed napisaniem pierwszej linijki kodu. Kluczowym narzędziem analitycznym jest **Modelowanie Zagrożeń** (Threat Modeling), w szczególności metodyka **STRIDE** opracowana przez firmę Microsoft:

- **S – Spoofing (Podszywanie się):** Czy napastnik może podać się za innego użytkownika lub usługę? (*Obrona: mocne uwierzytelnianie, certyfikaty mTLS, tokeny kryptograficzne*).
- **T – Tampering (Manipulacja danymi):** Czy przesyłane parametry lub dane w bazie mogą zostać zmodyfikowane w locie? (*Obrona: integralność, podpisy HMAC, szyfrowanie TLS*).
- **R – Repudiation (Wypieralność):** Czy użytkownik może wyprzeć się wykonanej operacji finansowej lub administracyjnej? (*Obrona: niezaprzeczalne logi audytowe, sygnatury*).
- **I – Information Disclosure (Ujawnienie informacji):** Czy dane wrażliwe wyciekają w odpowiedziach HTTP, logach lub komunikatach o błędach? (*Obrona: szyfrowanie, minimalizacja danych*).
- **D – Denial of Service (Odmowa usługi):** Czy aplikacja może zostać zablokowana przez wysycenie zasobów CPU, pamięci lub połączeń bazy? (*Obrona: Rate Limiting, timeouty, asynchroniczność*).
- **E – Elevation of Privilege (Eskalacja uprawnień):** Czy zwykły klient może wymusić uprawnienia administratora? (*Obrona: ścisła kontrola RBAC/ABAC na serwerze*).

5. Automatyzacja testów bezpieczeństwa: SAST, DAST, SCA

W nowoczesnym cyklu wytwórczym oprogramowania (*DevSecOps*) ręczny audyt bezpieczeństwa przeprowadzany raz w roku jest niewystarczający. Stosuje się triadę zautomatyzowanych narzędzi weryfikujących kod na różnych etapach:

5.1. 1. SAST (Static Application Security Testing)

Analiza statyczna bada kod źródłowy bez jego uruchamiania. Wyszukuje niebezpieczne wzorce składniowe, podatne wywołania funkcji systemowych oraz brak sanityzacji.

Narzędzie standardowe: Semgrep – nowoczesny, deklaratywny analizator kodu obsługujący dziesiątki języków.

```
# Skanowanie repozytorium pod katem oficjalnego zestawu reguł OWASP Top 10
semgrep scan --config "p/owasp-top-ten" ./src
```

Listing 1: Przykładowe uruchomienie skanera SAST Semgrep

5.2. 2. DAST (Dynamic Application Security Testing)

Analiza dynamiczna bada działającą aplikację z perspektywy czarnej skrzynki (Black-Box). Narzędzie wysyła spreparowane żądania HTTP, testując reakcję serwera na wstrzyknięcia, błędy nagłówków i podatności sesyjne.

Narzędzia standardowe: OWASP ZAP (Zed Attack Proxy) oraz Burp Suite

```
docker run -t zaproxy/zap-stable zap-baseline.py -t http://127.0.0.1:3000
```

Listing 2: Uruchomienie automatycznego skanowania DAST w kontenerze OWASP ZAP

5.3. 3. SCA (Software Composition Analysis)

Narzędzia SCA analizują pliki manifestów zależności (`requirements.txt`, `package.json`, pliki blokujące `lock`) oraz obrazy kontenerowe, porównując wersje zainstalowanych pakietów z publicznymi bazami podatności (CVE/NVD).

Narzędzia standardowe: Trivy, pip-audit, npm audit

```
# Weryfikacja paczek Pythona
pip-audit -r requirements.txt

```

```
# Skanowanie całego obrazu Docker pod katem podatnosci OS i bibliotek
trivy image moje-api:latest
```

Listing 3: Skanowanie zależności Pythona oraz obrazu kontenera

6. Fundamenty architektoniczne: Zasady Secure by Design

W podsumowaniu pierwszego modułu wykładowego warto wyróżnić cztery nadrzędne reguły projektowe:

1. **Zasada najmniejszych uprawnień (Principle of Least Privilege):** Każdy moduł, proces i użytkownik powinien posiadać wyłącznie takie uprawnienia, jakie są bezwzględnie konieczne do wykonania zadania. Baza danych aplikacji nie powinna łączyć się z uprawnieniami konta `root`.
2. **Obrona w głąb (Defense in Depth):** Bezpieczeństwo nigdy nie może opierać się na pojedynczej linii obrony. Walidacja po stronie klienta musi być powtórzona na serwerze, zabezpieczona zapytaniem parametryzowanym, a baza danych dodatkowo ograniczona prawami dostępu.
3. **Nigdy nie ufaj danym wejściowym (Never Trust User Input):** Wszystkie dane pochodzące z zewnątrz (parametry URL, nagłówki HTTP, ciasteczka, treść formularzy) muszą być traktowane jako potencjalnie złośliwe.
4. **Podejście Shift-Left:** Przesunięcie testów bezpieczeństwa maksymalnie w lewo na osi czasu projektu – od etapu pisania kodu i commitowania do repozytorium, a nie dopiero po wdrożeniu na środowisko produkcyjne.

7. Materiały dydaktyczne i repozytorium

Komplet materiałów dydaktycznych, prezentacji interaktywnych oraz instrukcji laboratoryjnych dostępny jest w oficjalnych kanałach przedmiotu:

- **Repozytorium Git (kod, prezentacje, przykłady):**
  `https://dev.zarnecki.org/Merito/bezpieczenstwo_aplikacji.git`
- **Chmura Nextcloud (materiały, skrypty PDF, archiwa zadań):**
  `https://chmura.zarnecki.org/s/iLdT3J5YGTTYCPr`